

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Вятский государственный университет»
(ВятГУ)
Колледж ВятГУ

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
«Восстановление системы (SQLite)»
«МДК 06.02 Инженерно-техническая поддержка сопровождения
информационных систем»

Выполнил: студент учебной группы
ИСПк-306-52-00
Боков А.С.

Преподаватель:
Самоделкин П.А.

Киров
2026

Цель работы

Изучение приёмов и методов организации непрерывной работы информационных систем, подлежащих защите. В рамках работы изучается встраиваемая система управления базами данных SQLite: развёртывание использующих её программ, анализ структуры баз данных и применяемых SQL-запросов, а также приёмы резервного копирования и восстановления базы данных.

Задание

Самостоятельно найти на сервисе GitHub и локально развернуть три программы, использующие СУБД SQLite. Проанализировать работу программ, изучить используемые в них SQL-запросы, рассмотреть структуру баз данных. Для работы с базами данных использовать программу DB Browser for SQLite. При поиске исключить проекты, использующие фреймворки FastAPI, Flask, Django, а также проекты с Docker-образами.

Результаты выполнения задания

1 Теоретические сведения

Система управления базами данных (СУБД) — комплекс программных средств, предназначенный для создания баз данных, хранения и обработки содержащихся в них данных. По модели хранения данных СУБД делятся на реляционные, в которых данные хранятся в виде связанных таблиц и обрабатываются языком SQL, и нереляционные (NoSQL), использующие иные модели хранения. Характеристики рассмотренных СУБД приведены в таблице 1.

Таблица 1 — Классификация и характеристики рассмотренных СУБД

СУБД	Тип	Архитектура	Особенности применения
SQLite	Реляционная	Встраиваемая, вся БД — один файл	Не требует сервера и настройки; применяется в мобильных и настольных приложениях, браузерах
MySQL	Реляционная	Клиент-серверная	Массово применяется на веб-сайтах; высокая скорость чтения
PostgreSQL	Объектно-реляционная	Клиент-серверная	Строгое соответствие стандарту SQL, сложные типы данных, транзакции
MongoDB	NoSQL, документная	Клиент-серверная	Хранит документы формата JSON, схема данных не фиксирована
Firebase	NoSQL, облачная	Облачный сервис	Синхронизация данных в реальном времени, применяется в мобильной разработке
Redis	NoSQL, «ключ — значение»	Сервер, данные в оперативной памяти	Очень высокая скорость; применяется для кеширования и очередей

Основные операции над данными обозначаются аббревиатурой CRUD. Соответствие этих операций командам языка SQL приведено в таблице 2.

Таблица 2 — Соответствие операций CRUD командам SQL

Операция CRUD	Команда SQL	Назначение
Create (создание)	INSERT	Добавление новой записи в таблицу
Read (чтение)	SELECT	Выборка записей из таблицы
Update (изменение)	UPDATE	Изменение значений полей существующих записей
Delete (удаление)	DELETE	Удаление записей из таблицы

Ключевая особенность SQLite, важная для организации непрерывной работы информационной системы, состоит в том, что вся база данных, включая схему и данные, размещается в одном файле на диске. Это позволяет выполнять резервное копирование базы обычным копированием файла, а восстановление — возвратом файла на прежнее место.

2 Отбор программ и порядок их развёртывания

Поиск программ выполнялся на сервисе GitHub. К отбираемым проектам предъявлялись следующие требования:

- язык программирования — Python, СУБД — SQLite (модуль sqlite3);
- отсутствие веб-фреймворков FastAPI, Flask, Django и Docker-образов в соответствии с условием задания;
- небольшой объём исходного кода, позволяющий полностью разобрать используемые SQL-запросы;
- наличие файла базы данных в репозитории либо создание базы при первом запуске.

Отобраны три программы, перечисленные в таблице 3. Программы различаются по объёму кода, стилю программирования и полноте реализации операций CRUD, что позволяет сравнить разные подходы к работе с одной и той же СУБД.

Таблица 3 — Программы, отобранные для анализа

№	Программа	Репозиторий GitHub	Объём кода
1	Expense Tracker (учёт расходов)	github.com/ SarthakSKumar/ Expense-Tracker- Application	89 строк
2	Contact Manager (записная книжка)	github.com/ motassimkhan/Contact- Manager	140 строк
3	Library Management System (библиотека)	github.com/ Muhammad-Ahmed- Rayyan/Library- Management-System	252 строки

Развёртывание каждой программы выполнялось по одинаковому порядку:

1. Клонирование репозитория командой `git clone`.
2. Установка недостающих внешних библиотек командой `pip install`.
3. Запуск программы командой `python <имя файла>.py`.
4. Проверка работы программы и содержимого базы данных.

Ход развёртывания третьей программы приведён на рисунке 1. Используемая версия ядра SQLite — 3.45.3, она входит в состав стандартного модуля `sqlite3` интерпретатора Python и отдельной установки не требует.

```

Windows PowerShell

PS C:\...\Лабораторная работа №1\Программы> git clone https://github.com/Muhammad-Ahmed-Rayyan/Library-Management-System.git
Cloning into 'Library-Management-System'...
remote: Enumerating objects: 29, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (29/29), done.
remote: Total 29 (delta 6), reused 4 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (29/29), 15.85 KiB | 2.26 MiB/s, done.
Resolving deltas: 100% (6/6), done.

PS ...\Программы> cd Library-Management-System

PS ...\Library-Management-System> dir

Mode                Length      Name
----                -
-a----          12288  library.db
-a----          10790  library.py
-a----           1156  LICENSE
-a----           2169  README.md

PS ...\Library-Management-System> pip install tkcalendar
Requirement already satisfied: tkcalendar in ...\Python312\Lib\site-packages (1.6.1)
Requirement already satisfied: babel in ...\Python312\Lib\site-packages (from tkcalendar) (2.18.0)

PS ...\Library-Management-System> python -c "import sqlite3; print('SQLite engine:', sqlite3.sqlite_version)"
SQLite engine: 3.45.3

PS ...\Library-Management-System> python library.py
(открывается графическое окно программы)

```

Рисунок 1 — Клонирование репозитория, установка зависимостей и запуск программы

3 Программа 1 — Expense Tracker (учёт расходов)

Программа предназначена для учёта личных расходов. Пользователь вводит дату, имя, назначение и сумму расхода, после чего запись сохраняется в базе данных. Кнопка View expenses выводит все сохранённые записи и их общую сумму. Интерфейс построен на стандартной библиотеке tkinter, для выбора даты используется внешняя библиотека tkcalendar (единственная зависимость, устанавливаемая командой `pip install tkcalendar`). Внешний вид программы после нажатия кнопки View expenses приведён на рисунке 2.

Рисунок 2 — Окно программы Expense Tracker с выведенными из базы данными

База данных хранится в файле `expenseTracker.db` и содержит одну таблицу `expenses`; структура таблицы приведена в таблице 4.

Таблица 4 — Структура таблицы `expenses`

Поле	Тип в коде	Назначение
date	string	Дата расхода
name	string	Имя лица, совершившего расход
title	string	Назначение расхода
expense	number	Сумма расхода

Все SQL-запросы программы приведены ниже.

```
CREATE TABLE IF NOT EXISTS expenses (
    date string, name string, title string, expense number)
INSERT INTO expenses VALUES (?, ?, ?, ?)
SELECT * FROM expenses
SELECT sum(expense) FROM expenses
```

Анализ работы программы. Таблица создаётся при каждом запуске программы, но благодаря конструкции `IF NOT EXISTS` повторного создания не происходит. Добавление записи выполняется параметризованным

запросом: вместо подстановки значений в текст запроса используются знаки вопроса, а сами значения передаются вторым аргументом метода `execute`. Такой способ является защитой от SQL-инъекций. Для подсчёта общей суммы расходов применяется агрегатная функция `sum`.

Из четырёх операций CRUD в программе реализованы только две — добавление (`INSERT`) и чтение (`SELECT`); изменение и удаление записей не предусмотрены. Кроме того, в описании таблицы указаны типы `string` и `number`, которых в SQLite не существует. Ошибки при этом не возникает: SQLite использует динамическую типизацию и определяет тип хранения по правилам сродства типов, относя оба указанных типа к текстовому и числовому сродству соответственно. Ни одно поле не объявлено первичным ключом, поэтому таблица допускает полные дубликаты записей.

4 Программа 2 — Contact Manager (записная книжка)

Программа предназначена для ведения списка контактов: хранит имя, адрес электронной почты и номер телефона, позволяет добавлять, изменять и удалять записи. В отличие от первой программы, код организован в виде класса, а все обращения к базе данных выполняются через единый метод `execute_db_query`. Внешний вид программы приведён на рисунке 3.

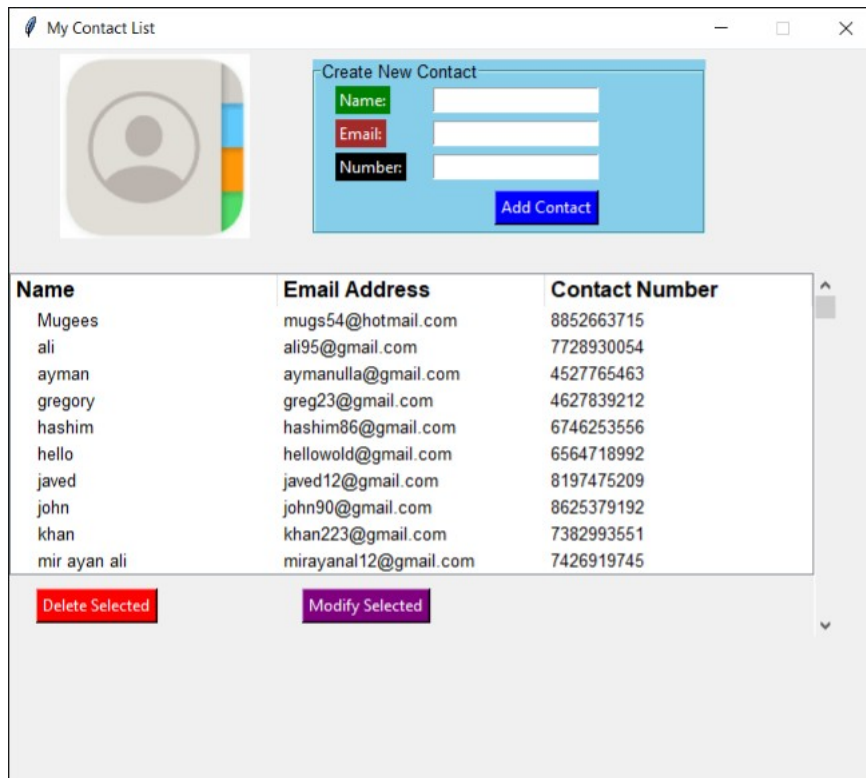


Рисунок 3 — Окно программы Contact Manager со списком контактов из базы данных

База данных хранится в файле contacts.db и содержит одну таблицу contacts_list; её структура приведена в таблице 5.

Таблица 5 — Структура таблицы contacts_list

Поле	Тип	Ограничения	Назначение
id	integer	PRIMARY KEY	Уникальный номер записи
name	varchar(255)	NOT NULL	Имя контакта
email	varchar(255)	NOT NULL, UNIQUE	Адрес электронной почты
number	integer	NOT NULL, UNIQUE	Номер телефона

SQL-запросы программы и соответствующие им операции CRUD приведены в таблице 6.

Таблица 6 — SQL-запросы программы Contact Manager

SQL-запрос	Операция	Назначение
INSERT INTO contacts_list VALUES(NULL,?, ?,?)	Create	Добавление контакта; значение NULL для поля id заставляет SQLite присвоить номер автоматически
SELECT * FROM contacts_list ORDER BY name desc	Read	Вывод всех контактов с сортировкой по имени в обратном порядке
UPDATE contacts_list SET number=? WHERE number =? AND name =?	Update	Изменение номера телефона выбранного контакта
DELETE FROM contacts_list WHERE name = ?	Delete	Удаление контакта по имени

Анализ работы программы. В программе реализованы все четыре операции CRUD. Структура таблицы описана значительно строже, чем в первой программе: задан первичный ключ, а ограничения NOT NULL и UNIQUE не позволяют оставить поля пустыми и создать два контакта с одинаковой почтой или одинаковым номером телефона. Соединение с базой открывается через контекстный менеджер with, что гарантирует корректное завершение транзакции. Все запросы параметризованы.

При этом в программе имеется существенный недостаток: удаление записи выполняется по полю name, а не по первичному ключу id. Поскольку ограничение уникальности на имя не наложено, при наличии двух контактов с одинаковым именем запрос DELETE удалит их одновременно. Корректным решением было бы удаление по условию WHERE id = ?.

5 Программа 3 — Library Management System (библиотека)

Программа предназначена для учёта книг библиотеки: хранит название книги, её идентификатор, автора, состояние (выдана или доступна) и номер читательского билета. Помимо операций CRUD в программе реализована дополнительная логика — выдача книги читателю и её возврат. Внешний вид программы приведён на рисунке 4.

Book Name	Book ID	Author	Status	Issuer Card ID
1984	BK003	George Orwell	Available	N/A
Harry Potter & The Order Of Phoe	BK007	JK. Rowling	Available	N/A
Harry Potter & The Prisoner of Azi	BK006	JK. Rowling	Available	N/A
Pride and Prejudice	BK005	Jane Austen	Available	N/A
The Great Gatsby	BK001	F. Scott Fitzgerald	Available	N/A
The Hobbit	BK004	J.R.R. Tolkien	Issued	C1002
To Kill a Mockingbird	BK002	Harper Lee	Issued	C1001

Рисунок 4 — Окно программы Library Management System с перечнем книг из базы данных

База данных хранится в файле library.db и содержит одну таблицу Library; её структура приведена в таблице 7.

Таблица 7 — Структура таблицы Library

Поле	Тип	Ограничения	Назначение
BK_NAME	TEXT	—	Название книги
BK_ID	TEXT	PRIMARY KEY, NOT NULL	Инвентарный номер книги
AUTHOR_NAME	TEXT	—	Автор книги
BK_STATUS	TEXT	—	Состояние: Available или Issued
CARD_ID	TEXT	—	Номер читательского билета

Основные SQL-запросы программы приведены в таблице 8.

Таблица 8 — SQL-запросы программы Library Management System

SQL-запрос	Назначение
CREATE TABLE IF NOT EXISTS Library (...)	Создание таблицы при первом запуске
SELECT COUNT(*) FROM Library	Проверка, пуста ли таблица; если пуста, заполняется примерами книг
INSERT INTO Library VALUES (?, ?, ?, ?, ?)	Пакетное добавление начальных записей методом executemany
SELECT * FROM Library ORDER BY BK_NAME	Вывод перечня книг с сортировкой по названию
UPDATE Library SET BK_NAME=?, BK_STATUS=?, AUTHOR_NAME=?, CARD_ID=? WHERE BK_ID=?	Изменение данных книги по инвентарному номеру
DELETE FROM Library WHERE BK_ID=?	Удаление одной книги
DELETE FROM Library	Полная очистка перечня книг

Анализ работы программы. Программа наиболее полно использует возможности СУБД. Поле BK_ID объявлено первичным ключом, поэтому добавление книги с уже существующим инвентарным номером невозможно: SQLite возбуждает исключение sqlite3.IntegrityError, которое программа перехватывает и выводит пользователю понятное сообщение об ошибке вместо аварийного завершения. Начальное заполнение таблицы выполняется методом executemany, добавляющим несколько записей одним вызовом. Изменение и удаление записей выполняются по первичному ключу, что исключает недостаток, отмеченный во второй программе.

6 Сравнительный анализ рассмотренных программ

Результаты сравнения трёх программ приведены в таблице 9.

Таблица 9 — Сравнение рассмотренных программ

Признак	Expense Tracker	Contact Manager	Library Management System
Объём кода	89 строк	140 строк	252 строки
Организация кода	Процедурный стиль	Класс, единый метод запросов	Функции и обработка ошибок
Файл базы данных	expenseTracker.db	contacts.db	library.db
Число таблиц	1	1	1
Первичный ключ	Отсутствует	id (integer)	BK_ID (text)
Ограничения целостности	Отсутствуют	NOT NULL, UNIQUE	PRIMARY KEY, NOT NULL
Операции CRUD	Create, Read	Все четыре	Все четыре
Параметризация запросов	Да	Да	Да
Обработка ошибок БД	Отсутствует	Отсутствует	Перехват IntegrityError

Сравнение показывает, что при использовании одной и той же СУБД надёжность хранения данных определяется тем, насколько полно разработчик применяет её средства. Наиболее защищённой от появления некорректных данных является третья программа, наименее — первая, где отсутствуют первичный ключ и какие-либо ограничения целостности.

7 Работа с базами данных в DB Browser for SQLite

Для непосредственного просмотра баз данных использовалась программа DB Browser for SQLite версии 3.13.1. Программа позволяет открыть файл базы данных, просмотреть перечень таблиц и их структуру на вкладке Database Structure, содержимое таблиц — на вкладке Browse Data, а также выполнить произвольный SQL-запрос на вкладке Execute SQL. Вид базы данных library.db, открытой в программе, приведён на рисунке 5. В строке Schema отображается команда создания таблицы, полностью совпадающая с той, что выполняется в исходном коде программы.

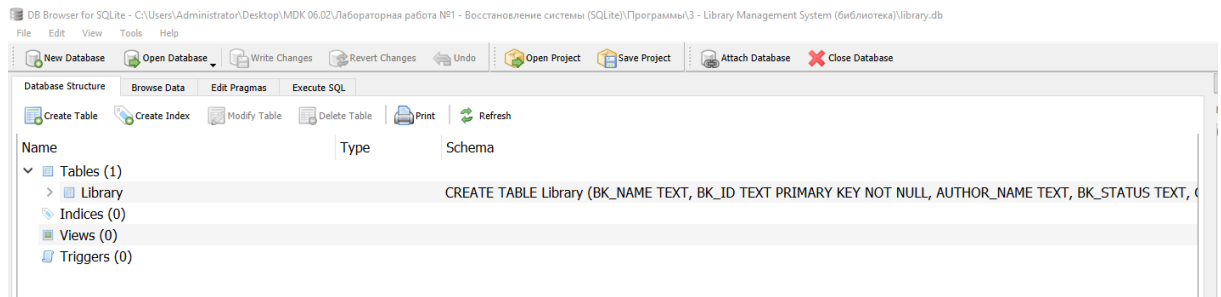


Рисунок 5 — База данных library.db, открытая в DB Browser for SQLite

8 Резервное копирование и восстановление базы данных

Так как база данных SQLite представляет собой один файл, резервное копирование выполняется его копированием, а восстановление — обратной заменой файла. Для проверки этого приёма был подготовлен сценарий backup_demo.py, выполняющий следующие действия: создание резервной копии файла базы, имитацию аварии — удаление всех записей запросом DELETE FROM Library, восстановление базы из резервной копии и проверку целостности восстановленного файла командой PRAGMA integrity_check. Результат выполнения сценария приведён на рисунке 6.

```
Windows PowerShell

PS ...\\Лабораторная работа №1> python backup_demo.py
1. Исходное состояние базы
   записей в таблице Library: 7

2. Создание резервной копии (копирование файла БД)
   создан файл library_backup.db, 12288 байт

3. Имитация аварии: удаление всех данных
   выполнен запрос: DELETE FROM Library
   записей в таблице Library: 0

4. Восстановление базы из резервной копии
   файл library_backup.db скопирован на место library.db
   записей в таблице Library: 7

5. Проверка целостности восстановленной базы
   PRAGMA integrity_check: ok
   первые записи: BK001 - The Great Gatsby; BK002 - To Kill a Mockingbird; BK003 - 1984

Данные восстановлены полностью.
```

Рисунок 6 — Резервное копирование, имитация аварии и восстановление базы данных

После удаления данных таблица содержала 0 записей, после восстановления из резервной копии — исходные 7 записей. Проверка целостности вернула результат ok, что подтверждает корректность восстановленного файла. Таким образом, регулярное копирование файла

базы данных обеспечивает возможность быстрого восстановления информационной системы после потери данных.

Выводы по работе

В ходе выполнения лабораторной работы изучены приёмы организации непрерывной работы информационных систем, использующих СУБД SQLite. На сервисе GitHub найдены и локально развёрнуты три программы, удовлетворяющие условиям задания: Expense Tracker, Contact Manager и Library Management System. Все программы написаны на языке Python с использованием стандартного модуля sqlite3 и библиотеки tkinter, веб-фреймворки и Docker в них не применяются.

Для каждой программы рассмотрена структура базы данных, изучены применяемые SQL-запросы и выполнен анализ работы. Установлено, что все три программы используют параметризованные запросы, защищающие от SQL-инъекций, однако полнота использования средств СУБД различается: в первой программе отсутствуют первичный ключ и ограничения целостности, а из операций CRUD реализованы только добавление и чтение; вторая и третья программы реализуют все четыре операции CRUD и используют ограничения целостности, причём третья программа дополнительно обрабатывает исключение sqlite3.IntegrityError при попытке добавить запись с уже существующим первичным ключом. Выявлен недостаток второй программы: удаление записи выполняется по имени контакта, а не по первичному ключу, что может привести к удалению нескольких записей одновременно.

Освоена работа с программой DB Browser for SQLite: просмотр структуры базы данных, содержимого таблиц и выполнение SQL-запросов. Изучены основные термины: понятие СУБД, различие реляционных и нереляционных баз данных, состав операций CRUD и соответствующие им команды SQL.

Практически проверен приём резервного копирования и восстановления базы данных SQLite: поскольку вся база хранится в одном файле, её копирование и восстановление выполняются средствами файловой системы, что подтверждено экспериментом с последующей проверкой целостности. Именно эта особенность делает SQLite удобным средством обеспечения сохранности данных и непрерывной работы информационных систем небольшого масштаба.

Цель работы достигнута, задание выполнено в полном объёме.